

教育助手AI Agent - 产品需求与分阶段开发任务设计文档 (PRD)

版本: V4 | 定位: 面向研发团队的详细实施规范

一、项目概述与架构准则

本项目旨在构建一个基于 Java 生态与 Spring AI 的教育领域 AI Agent。系统需实现教育任务的意图拆解、自动化工具调用及质量自审。整体设计遵循高内聚低耦合、客观务实、逻辑可追溯的工程原则，摒弃过度设计，优先保障后端状态机流转与大模型交互链路的稳定性。

二、阶段一：基础设施、数据底座与通信通道 (MVP 基础)

需求目标：建立可靠的任务流转状态机与前后端异步通信机制。此阶段暂不接入真实大模型，通过 Mock 数据跑通整个系统骨架。

2.1 核心需求描述

- 实现任务生命周期管理 (PENDING -> PLANNING -> EXECUTING -> REVIEWING -> COMPLETED/FAILED)。
- 实现 Web 前端基础页面，支持 SSE (Server-Sent Events) 单向流式数据接收。
- 设计并落地 MySQL 核心业务表，保障任务轨迹100%落库。

2.2 具体开发任务分配

任务编号	模块名称	开发任务明细
DEV-1.1	数据库设计与 DDL	1. 编写 edu_task (任务主表) 和 edu_task_log (执行轨迹表) 的 DDL。 2. 引入 MyBatis-Plus 或 Spring Data JPA 完成实体映射与基础 CRUD Mapper。
DEV-1.2	REST API 与 SSE 接口	1. 开发 POST /api/task/create 接口, 接收指令, 初始化状态机, 返回 TaskId。 2. 开发 GET /api/task/stream/{taskId} SSE 接口, 利用 SseEmitter 实时推送模拟的任务状态变更日志至前端。
DEV-1.3	基础前端交互(可选)	编写极简的前端页面, 包含输入框与控制台风格的输出框, 验证 SSE 消息能否实时打印。

三、阶段二: Spring AI 接入与 Tool 工具链编排 (核心逻辑)

需求目标: 正式引入 Spring AI 依赖, 完成大语言模型的对接。实现基于 Prompt 工程的任务拆解(Planner)以及本地 Java 工具(Function Calling)的注册与调度。

3.1 核心需求描述

- Agent 接收到指令后, 必须强制输出 JSON 格式的拆解步骤(Step-by-Step)。
- 系统需能够根据步骤, 动态反射调用对应的本地工具。
- 首期实现两个本地工具: 文本摘要生成、标准化习题生成。

3.2 具体开发任务分配

任务编号	模块名称	开发任务明细
DEV-2.1	ChatClient 配置	1. 在 application.yml 中配置模型参数与 API Key。 2. 编写全局 System Prompt, 定义模型角色为“客观、严谨的教育任务规划引擎”。
DEV-2.2	JSON 结构化输出拦截	1. 使用 Spring AI 的 BeanOutputConverter 机制, 要求大模型输出包含 steps 数组的指定 JSON 结构。 2. 编写反序列化逻辑, 将输出映射为 Java 的 TaskPlan 实体对象。
DEV-2.3	Tool 函数注册	1. 开发 TextSummaryTool 并使用 @Bean 结合 @Description 注入容器。 2. 开发 QuizGeneratorTool, 定义严格的入参和出参记录。

四、阶段三：多模态摄入与 Milvus 混合检索架构 (RAG 增强)

需求目标:解决长文本教育资料输入及专业名词幻觉问题。引入 OCR 及 Milvus 向量数据库, 构建高精度的 RAG(检索增强生成)流水线。

4.1 核心需求描述

- 支持用户上传 PDF 或图片, 系统自动提取文本作为初始 Context。
- 在本地知识检索工具中, 弃用单一语义检索, 全面改用稠密向量(Dense)与稀疏向量(Sparse)的混合检索方案, 提高召回率与准确度。

4.2 具体开发任务分配

任务编号	模块名称	开发任务明细
DEV-3.1	文档预处理组件	1. 引入 Apache PDFBox 提取 PDF 纯文本流。 2. 将长文本按照固定 Token 长度(如 512)与 overlap 进行 Chunking 分块。
DEV-3.2	Milvus 集合创建与写入	1. 定义 Milvus Collection Schema: 包含 Chunk_ID、Text_Content、Dense_Vector (Float) 以及 Sparse_Vector (Sparse Float)。 2. 开发入库逻辑: 调用 Embedding 模型生成稠密向量, 并生成稀疏关键字向量后批量 Insert。
DEV-3.3	Hybrid Search 查询实现	1. 扩展知识检索 Tool, 接收查询关键词。 2. 调用 Milvus 的 runHybridSearch 接口, 并发执行 Dense 与 Sparse 查询。 3. 在 Java 代码层面实现倒数秩融合(RRF)算法, 重排结果提取 Top-K, 组装为高优 Prompt Context。

五、阶段四: 多 Agent 协同网络与质量把控 (防劣化机制)

需求目标: 建立内容质量防火墙。由单 Agent 升级为主从 Agent 架构, 形成“生成-审核-修正”的自我迭代闭环。

5.1 核心需求描述

- 引入 Critic Agent(审核者), 对 Generator Agent(生成者)的结果进行多维度评估。
- 如果评估不达标, 必须将修改建议回传给 Generator 进行重新生成, 且需防止无限死循环。

5.2 具体开发任务分配

任务编号	模块名称	开发任务明细
DEV-4.1	Critic 评审器实例开发	1. 构建独立 ChatClient 实例, 加载评审专用 Prompt。 2. 约束 Critic 返回格式为: {"passed": boolean, "suggestions": "..."}。
DEV-4.2	重试循环与熔断机制	1. 在主控服务层实现 while 或递归逻辑, 截获 Critic 建议并追加至 Generator 历史上下文中重试。 2. 硬编码最大重试次数(如 MAX_RETRY = 3)。超限则强制打断, 记录 FAILED 状态, 并将已有的可用产物兜底返回给前端。